

Universidade Cidade de São Paulo - UNICID
Curso de Ciência da Computação

ERIC CONDE DERRE
ERICK DO CARMO ESTEVES
FERNANDO MATOS DE SOUZA
GUILHERME PENHA DOS SANTOS
JOÃO VICTOR FONSECA SILVA
KENNEDY FERNANDO DE OLIVEIRA GUNDIM
LEANDRO HENRIQUE DA SILVA PATRICIO
PEDRO HENRIQUE TORRES GONÇALVES
RAUL SILVESTRE MARINHO

DESENVOLVIMENTO DE UM AGENTE DE QA END-TO-END (E2E)
AUTÔNOMO COM CAPACIDADES DE SELF-HEALING:
Focado na Fronteira de Integração

São Paulo
2026

ERIC CONDE DERRE
ERICK DO CARMO ESTEVES
FERNANDO MATOS DE SOUZA
GUILHERME PENHA DOS SANTOS
JOÃO VICTOR FONSECA SILVA
KENNEDY FERNANDO DE OLIVEIRA GUNDIM
LEANDRO HENRIQUE DA SILVA PATRICIO
PEDRO HENRIQUE TORRES GONÇALVES
RAUL SILVESTRE MARINHO

**DESENVOLVIMENTO DE UM AGENTE DE QA END-TO-END (E2E)
AUTÔNOMO COM CAPACIDADES DE SELF-HEALING:**

Focado na Fronteira de Integração

Projeto de pesquisa apresentado ao curso de
Ciência da Computação da Universidade Cidade
de São Paulo (UNICID), como requisito parcial
para a elaboração do Trabalho de Conclusão de
Curso.

São Paulo

2026

SUMÁRIO

1 INTRODUÇÃO

1.1 Contextualização e Delimitação do Tema

1.2 Problema de Pesquisa

1.3 Justificativa

1.4 Objetivos

2 REFERENCIAL TEÓRICO

2.1 Evolução da Automação E2E e o Conceito de Self-Healing

2.2 Fundamentos de Aprendizado de Máquina e Adaptação

2.3 Automação E2E e Inteligência Artificial

2.4 Gestão de Context Window em LLMs

2.5 Detecção de Quebra de Contrato de API

2.6 Recuperação de Conhecimento Prévio Aplicada à Localização de Falhas

3 METODOLOGIA

3.1 Infraestrutura e Orquestração Local

3.2 Inteligência Artificial e Processamento LLM

3.3 Comunicação e Mitigação de Gargalos

3.4 Validação e Avaliação

4 RESULTADOS ESPERADOS

5 CRONOGRAMA DE ATIVIDADES

REFERÊNCIAS

GLOSSÁRIO

1 INTRODUÇÃO

1.1 Contextualização e Delimitação do Tema

A automação de testes End-to-End (E2E) tradicional enfrenta um alto custo de manutenção devido à fragilidade crônica de seus scripts. Uma alteração simples no CSS do Frontend ou a mudança invisível do tipo de um dado retornado por uma API, fenômeno conhecido como *Contract Drift* (Deriva de Contrato), quebra instantaneamente a esteira de CI/CD (Integração Contínua/Entrega Contínua), gerando perda de tempo em triagens manuais exaustivas.

Embora a automação moderna baseada em Inteligência Artificial ofereça avanços, a literatura e a prática industrial demonstram que a IA não tem o intuito de substituir os engenheiros de qualidade (QA) ou os desenvolvedores, mas sim assumir as tarefas repetitivas. Dessa forma, os profissionais mantêm o controle sobre decisões de risco, estratégias de cobertura e testes exploratórios complexos.

Neste escopo, o presente projeto delimita-se ao desenvolvimento de um Agente de QA End-to-End (E2E) autônomo com capacidades de *Self-Healing* (autocura), focado estritamente na fronteira de integração (Front-to-Back) da API.

1.2 Problema de Pesquisa

Diante do cenário de fragilidade dos testes automatizados e das limitações de infraestrutura corporativa, levanta-se a seguinte questão de pesquisa: Como um Modelo de Linguagem Grande (LLM), aliado à interceptação bidimensional, pode diagnosticar autonomamente falhas na fronteira de integração Front-to-Back, sem a necessidade de acoplamento à lógica interna do servidor, operando sob restrições de hardware local?

1.3 Justificativa

Este trabalho justifica-se pela necessidade de desenvolver ferramentas de assistência focadas em acelerar a Análise de Causa Raiz para as equipes técnicas. Tradicionalmente, quando ocorre uma falha silenciosa na integração cliente-servidor, perdem-se horas cruzando capturas de tela com logs de servidor para entender o desvio.

O agente proposto atuará como um copiloto de depuração inteligente: ao unir a leitura estruturada da interface renderizada à interceptação nativa de rede do framework Playwright, ele pré-processa a falha e entrega ao desenvolvedor um diagnóstico rápido de onde o contrato da API divergiu do esperado pela interface. O projeto visa melhorar o tempo de resposta da equipe frente a bugs de API e reduzir o desgaste com a manutenção de rotinas frágeis, permitindo que o profissional foque na revisão da qualidade e na garantia da integridade da arquitetura de software.

1.4 Objetivos

Objetivo Geral: Desenvolver, implementar e mensurar a eficácia de um Agente de QA E2E Autônomo, utilizando um LLM executado localmente para realizar o *Self-Healing* dos localizadores de scripts de teste e para produzir o diagnóstico de causa raiz a partir da análise conjunta da interface e da interceptação de rede, facilitando a depuração pela equipe.

Objetivos Específicos:

- Levantar a fundamentação teórica e preparar o ambiente base, pesquisando metodologias de automação com LLMs, extração de árvores de acessibilidade e quebra de contrato de APIs.
- Prototipar a camada de inferência local, configurando o orquestrador e o runtime de modelos (Ollama) para executar o modelo Qwen2.5-Coder quantizado integralmente na máquina do usuário, sem dependência de infraestrutura em nuvem.
- Implementar o Módulo de Filtragem de Contexto (DOM Pruning) para impedir o estouro da janela de contexto (Context Window) do LLM.
- Desenvolver o Agente de Interceptação e Self-Healing, integrando a automação visual para capturar requisições HTTP e codificar a lógica autônoma de correção.
- Mensurar e validar a precisão e performance do agente contra aplicações de teste, utilizando injeção de falhas sintéticas e Fuzzing de mutação de API.

2 REFERENCIAL TEÓRICO

Este projeto fundamenta-se nos avanços documentados na literatura técnico-científica sobre a adoção de modelos generativos no ciclo de vida de desenvolvimento de software (SDLC) e nos fundamentos clássicos de aprendizado de máquina.

2.1 Evolução da Automação E2E e o Conceito de Self-Healing

A automação de testes de software transitou de scripts manuais lineares para abordagens complexas de ponta a ponta (End-to-End - E2E), projetadas para simular o comportamento de um usuário real em toda a pilha tecnológica da aplicação. Inicialmente, a automação E2E baseava-se em localizadores estáticos dentro do Document Object Model (DOM) — a estrutura HTML da página. Contudo, essa dependência tornava os testes frágeis e suscetíveis a quebras mediante qualquer alteração visual ou estrutural na interface gráfica.

Para mitigar este problema, o conceito de autocura (*Self-Healing*) emergiu na interseção entre engenharia de software e inteligência artificial. Em sistemas distribuídos, a autocura refere-se à capacidade de um sistema detectar falhas ativamente, diagnosticar sua causa raiz e aplicar correções de forma autônoma. No contexto de testes de qualidade (QA), um agente dotado de *Self-Healing* utiliza algoritmos de busca e processamento de linguagem natural para identificar dinamicamente novos localizadores na interface ou interpretar desvios nas respostas do servidor (*Contract Drift*) quando o script original falha, restaurando assim o fluxo de execução sem a necessidade de intervenção humana imediata.

Cabe delimitar o grau de autonomia adotado neste trabalho. A correção de localizador é gerada e **aplicada automaticamente** pelo agente em uma reexecução de verificação, o que permite medir objetivamente se o fluxo do teste foi de fato restaurado (Task Success). Contudo, a alteração não é incorporada de forma definitiva ao repositório de testes sem revisão: ela é registrada e **apresentada ao desenvolvedor como sugestão**. Essa delimitação preserva o caráter de copiloto descrito na justificativa, sem abrir mão da verificação automática necessária à avaliação quantitativa.

2.2 Fundamentos de Aprendizado de Máquina e Adaptação

A transição de sistemas baseados em regras estritas para sistemas adaptativos (como os testes com *Self-Healing*) encontra sua base teórica no aprendizado de máquina. Segundo Faceli et al. (2025), os algoritmos de aprendizado de máquina permitem que sistemas computacionais extraiam padrões intrínsecos a partir de dados para a tomada de decisão autônoma, superando as limitações da programação puramente determinística. Ao invés de falhar imediatamente ante uma mudança no DOM, o sistema dotado de inteligência

artificial utiliza o reconhecimento de padrões para inferir a intenção original do teste e sugerir a correção.

A mesma obra fornece o instrumental de avaliação adotado neste projeto. O Capítulo 10, dedicado à avaliação de modelos preditivos, trata da partição dos dados entre treino e teste, da matriz de confusão e das métricas dela derivadas — acurácia, precisão, revocação e F1 —, bem como de suas médias macro e micro em problemas de classificação multiclasse, que é a forma assumida pela tarefa de diagnóstico deste trabalho: atribuir à falha observada uma entre várias causas raiz possíveis. O Capítulo 7, sobre métodos connexionistas, cobre as redes neurais profundas que servem de base à arquitetura de transformadores empregada pelos LLMs, situando o modelo de linguagem utilizado no mesmo arcabouço teórico (FACELI et al., 2025 — capítulos identificados pelo sumário da 3. ed.; conteúdo a conferir no exemplar).

2.3 Automação E2E e Inteligência Artificial

Estudos recentes, como o GenIA-E2ETest (JÚNIOR et al., 2025), compõem a base metodológica que atesta a eficácia dos LLMs (Modelos de Linguagem Grandes) na interpretação de intenções textuais e tradução para comandos automatizados de interação de elementos de interface. A implementação do Playwright Healer Agent (BISWAS, 2026\)) sustenta o diferencial arquitetural deste trabalho, validando que a interceptação do tráfego de rede aliada à IA mitiga drasticamente os falsos-positivos.

2.4 Gestão de Context Window em LLMs

A literatura comprova que sobrecarregar a memória de contexto (Context Window) dos LLMs com códigos HTML puros degrada seu raciocínio lógico, um fenômeno conhecido como Attention Dilution. Metodologias focadas em podar e limpar a árvore de elementos antes do processamento, como o Prune4Web (ZHANG, J. et al., 2026), formam a espinha dorsal das técnicas de filtragem utilizadas neste projeto para viabilizar inferências precisas com custo computacional reduzido.

Complementarmente, Joseph (2026) demonstra que a extração da árvore de acessibilidade (*Accessibility Tree*) constitui, por si só, uma estratégia de redução de contexto de baixo custo, por expor apenas a projeção semântica dos elementos — seu papel (*role*), nome acessível e valor — em vez da árvore HTML completa. Este projeto adota essa abordagem como substrato: a poda é aplicada **sobre a árvore de acessibilidade**, e não sobre o DOM bruto, que passa a ser consultado apenas pontualmente, durante a resolução de um localizador na etapa de autocura.

O limite dessa janela, contudo, não é contado em caracteres, e sim em *tokens* — as unidades discretas em que o texto é convertido pela tokenização, etapa que antecede

qualquer processamento pelo modelo. A correspondência entre caracteres e tokens, e a fertilidade (tokens por palavra) que dela decorre, variam conforme o idioma e o tokenizador empregado. Para o português, Corrêa et al. (2026) medem, em corpus de cerca de 600 mil palavras, 2,88 caracteres por token no tokenizador nativo do modelo Tucano 2 e entre 2,71 e 2,72 caracteres por token nos tokenizadores de três modelos abertos recentes — faixa compatível com os 2,6 a 2,8 caracteres por token apurados nos ensaios preliminares deste projeto com os tokenizadores reais dos modelos avaliados. A diferença entre tokenizadores não é marginal: segundo Corrêa et al. (2024), o mesmo texto de 7.400 palavras em português consumiu 9.937 tokens com o tokenizador do TeenyTinyLlama e 14.813 com o do Sabiá-7B. A consequência prática é que o orçamento de contexto não pode ser fechado pela regra prática de estimar um token a cada quatro caracteres, formulada para o inglês: a estimativa por caracteres serve para montar o prompt, e é a contagem real de tokens devolvida pelo runtime depois de cada inferência que confirma se o orçamento foi respeitado.

Disso decorre a distinção entre a janela nominal e a janela efetivamente útil. Quando o prompt excede o limite configurado, o runtime Ollama descarta silenciosamente o início da conversa, sem qualquer campo de aviso na resposta de sua interface de programação, registrando o corte apenas em log de depuração (KULIGOWSKI, 2026) — e é justamente no início do prompt que ficam as instruções e o esquema da resposta esperada. Ainda que o conteúdo caiba na janela, a posição em que é colocado altera o desempenho. Liu et al. (2024) medem uma curva em U de aproveitamento no GPT-3.5-Turbo: com vinte documentos fornecidos ao modelo, o acerto é de 75,8% quando o documento relevante está na primeira posição, cai para 53,8% quando está na décima posição, no meio da lista, e sobe para 63,2% na última — no meio do contexto, o desempenho fica abaixo dos 56,1% obtidos pelo mesmo modelo sem contexto algum. Cuconasu et al. (2024) reproduzem o efeito em modelos abertos: no Llama2-7B, com um documento correto e 18 distratores, a acurácia é de 0,3781 quando o documento correto está junto à pergunta e de 0,1795 quando está no meio do contexto, comportamento consistente também em Falcon-7B, MPT-7B e Phi-2 (2,7B). As duas constatações — corte silencioso no início e perda no meio — sustentam duas providências adotadas neste projeto: ordenar o prompt do conteúdo mais estável para o mais variável e estimar, no início de cada época, o consumo de tokens do maior prompt da época e conferir a contagem real devolvida pelo runtime depois de cada inferência — um caso que ultrapasse o orçamento interrompe a época em vez de ser truncado em silêncio —, em vez de confiar na capacidade nominal declarada pelo runtime.

2.5 Detecção de Quebra de Contrato de API

A verificação automatizada de contratos entre cliente e servidor é discutida em texto técnico da InstaTunnel (2026), que caracteriza a deriva de contrato (*API drift*) como uma

classe de falha que escapa aos testes unitários de cada lado isoladamente, por se manifestar apenas na fronteira de integração. O texto destaca que alterações silenciosas de tipo, remoção de campos e renomeações são detectáveis por comparação estrutural da resposta observada contra o contrato esperado — princípio que fundamenta o gatilho de erro adotado neste projeto, no qual a interceptação de rede compara a forma (chaves e tipos) do dado recebido com a esperada pela interface consumidora.

2.6 Recuperação de Conhecimento Prévio Aplicada à Localização de Falhas

Shi, Li e Chen (2025) demonstram que a Geração Aumentada por Recuperação (*Retrieval-Augmented Generation* - RAG), quando sensível à funcionalidade do trecho analisado, melhora a localização de falhas por LLMs ao fornecer ao modelo apenas o conhecimento pertinente ao defeito em questão, em vez de ampliar indiscriminadamente o contexto. Neste trabalho, o princípio é aplicado por meio de uma base de conhecimento curada sobre a aplicação-alvo — organizada por módulo e por entidade de negócio, e contendo regras de negócio e defeitos previamente conhecidos — recuperada seletivamente no momento do diagnóstico.

Cabe ressaltar que essa base é **estática e construída previamente**, de forma análoga ao conhecimento acumulado por um analista de QA sobre o sistema que testa. Ela não implica inspeção da lógica interna do servidor em tempo de execução, preservando a delimitação de escopo estabelecida na seção 1.2.

A recuperação, porém, transfere ao material recuperado parte da responsabilidade pelo diagnóstico, e a literatura mede o custo de recuperar material incorreto. Wu, Wu e Zou (2024), em seis modelos de ponta e mais de mil e duzentas perguntas distribuídas por seis domínios, observam que os modelos abandonam conhecimento interno correto e adotam o conteúdo recuperado incorreto em mais de 60% dos casos, com adesão tanto maior quanto menor a confiança do modelo na própria resposta. Pelo lado da segurança, Zou et al. (2025) mostram que bastam cinco textos maliciosos por pergunta-alvo, inseridos em uma base com milhões de textos, para induzir em 90% dos casos a resposta escolhida pelo atacante. Daí a exigência, adotada neste trabalho, de que todo item da base de conhecimento — tanto os escritos à mão quanto os que venham a ser acrescentados depois — passe por uma conferência determinística, escrita em código, e não pelo julgamento do próprio modelo, antes de ficar disponível para recuperação.

Uma extensão natural dessa base é deixar que o próprio modelo a mantenha, acrescentando o que apura ao longo das execuções sem que seus pesos sejam alterados. Suzgun et al. (2025) demonstram que essa forma de aprendizado em tempo de teste produz ganho expressivo em modelos grandes, mas registram o limite que interessa diretamente a este projeto: em modelos menores os ganhos são atenuados por falta de competência para

escrever bons registros, e o desempenho chega a cair quando material de baixa qualidade é recuperado — o gargalo está em quem escreve, e não em quem lê. Zhang, G. et al. (2026) quantificam o efeito de um portão de admissão calculado majoritariamente em código, que rejeitou 63,1% dos itens candidatos e elevou o F1 da tarefa de 0,541 para 0,583 com 31% menos latência, sendo que quatro dos cinco sinais avaliados são regras determinísticas que custam menos de 65 ms, contra 2.580 ms do único sinal julgado por outro modelo de linguagem. É esse arranjo — base mantida pelo modelo, com a admissão de cada item decidida em código — que a etapa experimental descrita na seção 3.2 submete a teste.

3 METODOLOGIA

A pesquisa caracteriza-se como um projeto prático e aplicado (Pesquisa-Ação), voltado para a construção e validação de uma arquitetura de software escalável.

A execução organiza-se em etapas numeradas, cada uma ligada a um objetivo específico da seção 1.4: a Fase 1 prepara o ambiente e as duas aplicações-alvo (primeiro objetivo); as Fases 2-A, 2-B e 3 comparam os modelos de linguagem locais — primeiro sem documentação, depois com uma base de conhecimento recuperada e, por fim, com a base editada pelo próprio modelo — e decidem o modelo e o estado da base a adotar (segundo objetivo); a Fase 4 implementa a filtragem de contexto, a interceptação de rede e a autocura de localizadores (terceiro e quarto objetivos); e a Fase 5 mede o MTTR e o Task Success contra a linha de base manual (quinto objetivo). A infraestrutura baseia-se nas seguintes camadas:

3.1 Infraestrutura e Orquestração Local

A ferramenta principal de orquestração será o Playwright (com Node.js/Python). O Playwright comunica-se com o navegador via protocolos nativos e possui suporte embutido para interceptação e modificação de chamadas de rede em tempo real. Isso permite extrair a árvore de acessibilidade do DOM e escutar os códigos de status HTTP em segundo plano. Como contingência, o framework Cypress poderá ser acionado caso ocorram incompatibilidades críticas.

3.2 Inteligência Artificial e Processamento LLM

O processamento semântico utilizará o modelo Qwen2.5-Coder em variante quantizada, executado **integralmente na máquina local** por meio do runtime Ollama. A quantização reduz a precisão numérica dos pesos do modelo, diminuindo o consumo de memória a ponto de viabilizar a inferência em CPU, sem GPU dedicada.

A escolha do porte do modelo (1,5B, 3B ou 7B de parâmetros) será definida experimentalmente, por comparação entre as variantes sob o mesmo hardware, considerando latência por inferência, pico de memória e qualidade do diagnóstico produzido. Adota-se como critério o **menor modelo que atenda à tarefa**, e não o maior que couber na máquina, uma vez que o agente compete por memória com o navegador automatizado, o servidor de aplicação e o banco de dados da aplicação-alvo durante a execução dos testes.

O regime de base de conhecimento mantida pelo próprio modelo, discutido na seção 2.6, será submetido a teste em uma etapa experimental dedicada — aqui designada **Fase 3** —, na qual cada modelo avaliado recebe uma cópia própria da base, permanecendo a base

original intocada como ponto de partida comum a todos. A edição autorizada é apenas por acréscimo — nota em um item existente, retificação que aponte o trecho equivocado ou item novo —, nunca por remoção, de modo que toda alteração permaneça auditável pela comparação entre versões sucessivas. Nenhuma proposta de edição é incorporada sem passar antes por uma conferência escrita em código, e não pelo julgamento de outro modelo de linguagem, pelo motivo exposto na seção 2.6. O ciclo é repetido por três épocas, guardando-se a cópia da base ao final de cada uma, para que o desempenho possa ser lido época a época e não apenas ao final. Os casos são divididos em duas partições — 54 de aprendizado e 36 de avaliação —, e somente nos 54 o modelo tem acesso à causa correta e pode propor edições, enquanto os 36 nunca recebem retorno, o que separa o que o modelo generaliza do que ele apenas memoriza.

3.3 Comunicação e Mitigação de Gargalos

Com a inferência executada localmente, a comunicação entre o orquestrador e o modelo ocorre pela interface HTTP do próprio runtime, restrita à interface de loopback da máquina, dispensando exposição à rede externa e qualquer serviço de tunelamento. O framework FastAPI permanece em uso no projeto, porém no papel de API da aplicação-alvo submetida aos testes, e não como camada de integração com o modelo.

Para evitar o colapso da janela de contexto do LLM, aplicar-se-ão heurísticas estritas de poda sobre a árvore de acessibilidade (seção 2.4), enviando apenas os elementos semanticamente relevantes da interface, acompanhadas do truncamento dos dados JSON extensos interceptados na rede.

O truncamento observará uma restrição específica: reduz-se o **volume** do dado — encurtando listas extensas e cadeias de texto longas —, preservando-se integralmente suas **chaves e tipos**. Essa distinção é necessária porque a quebra de contrato manifesta-se na forma do dado, e não em sua quantidade; um truncamento que suprimisse campos eliminaria o próprio fenômeno que se pretende detectar.

3.4 Validação e Avaliação

A avaliação evitará a subjetividade inerente às análises feitas puramente por IAs, adotando aplicações-alvo determinísticas. As quebras de contrato serão simuladas, de forma dominante, por injeção determinística de falhas por modo: a aplicação-alvo expõe modos de falha fixos, e cada cenário é gravado como caso fixo, de modo que o mesmo caso seja idêntico entre modelos e entre rodadas; técnicas de Fuzzing (injeção de dados anômalos e imprevistos) e Mutação Dinâmica ficam reservadas a fases de teste posteriores, como complemento, e não como base da comparação. O sucesso do agente será avaliado quantitativamente por duas métricas principais: Tempo Médio de Reparo (MTTR) e Task

Success (Sucesso da Tarefa), verificando na prática se a sugestão refatorada consegue restaurar a estabilidade do teste.

Aplicações-alvo. Serão utilizadas duas aplicações de características deliberadamente distintas, ambas construídas e controladas pelo próprio grupo: uma aplicação monolítica legada, com renderização no servidor e acesso direto ao banco de dados, e uma interface moderna que consome uma API JSON por requisições assíncronas. A primeira exercita a abrangência do agente frente a sistemas legados; a segunda concentra os cenários de quebra de contrato, objeto central do trabalho.

Definições operacionais. O Tempo Médio de Reparo (MTTR) é medido como o intervalo entre a detecção da falha pelo agente e a restauração verificada do fluxo de execução, isto é, o instante em que a correção proposta é validada com sucesso na reexecução. O Task Success é definido como a proporção de cenários em que o fluxo foi restaurado e o roteiro de teste concluiu sem erro, sobre o total de cenários de falha injetados.

Linha de base. Como grupo de controle, os mesmos cenários serão resolvidos manualmente por integrantes do grupo, com o tempo cronometrado desde a apresentação da falha até a correção do teste. A comparação entre esse tempo e o MTTR do agente constitui o resultado central da avaliação.

Volume e determinismo. Cada configuração será submetida a cerca de dez cenários distintos, com cinco repetições cada, cobrindo os modos de falha injetados e a quebra de localizador. Para assegurar a reprodutibilidade, o banco de dados da aplicação-alvo é restaurado ao estado inicial entre execuções, o modo de falha é fixado por cenário, e a versão do navegador é mantida constante pelo próprio orquestrador, evitando variação decorrente de atualizações automáticas. Na Fase 3, cuja unidade de medida é o diagnóstico e não a reexecução de um roteiro, a reprodutibilidade foi medida de outro modo: uma execução por condição, com temperatura 0,1 e sem semente fixa, e a comparação caso a caso entre versões do runtime de inferência, em que nenhum rótulo trocou em seis dos oito pareamentos realizados (achado 4.31 do Memorial de Desenvolvimento do projeto).

Métricas da Fase 3. A métrica primária de comparação entre modelos e entre versões da base de conhecimento será a acurácia balanceada, isto é, a média das revocações obtidas em cada classe. A escolha é justificada, e não herdada: Opitz (2024) mostra que acurácia, F1 micro, macro e ponderada, kappa e coeficiente de correlação de Matthews diferem sistematicamente em viés e em sensibilidade à prevalência das classes, de modo que a métrica principal precisa ser argumentada e acompanhada das demais; e Collot et al. (2026) demonstram que a acurácia balanceada é a métrica alinhada à comparação de modelos sob desbalanceamento, por não depender da escolha arbitrária de uma classe

positiva — propriedade decisiva aqui, porque o modo de falha previsto é o colapso do modelo em um único rótulo, que a acurácia simples não distingue de acerto genuíno.

Inferência estatística. Toda proporção será reportada com intervalo de confiança de 95% pelo método de escore de Wilson, e não pelo intervalo de Wald derivado do Teorema Central do Limite: Bowyer, Aitchison e Ivanova (2025) medem que, com cerca de cem itens, o intervalo de Wald entrega apenas 92,5% de cobertura real, degradando severamente em amostras menores, a ponto de produzir intervalos de largura zero ou fora do intervalo $[0,1]$. Como todos os contrastes incidem sobre os mesmos casos, a comparação entre duas condições usará o teste de McNemar pareado, que aproveita o pareamento como fonte de potência estatística. Para as quatro versões da base geradas ao longo das épocas (L0 a L3), o teste conjunto será o Q de Cochran — a generalização do McNemar para mais de duas condições pareadas —, seguido, quando significativo, de comparações par a par com correção de Holm, procedimento passo a passo que García e Herrera (2008) apontam como mais potente que as alternativas conservadoras de uso corrente (a leitura do artigo quanto ao conjunto exato de procedimentos comparados permanece a conferir). Ao lado de cada valor-p será reportado o tamanho de efeito pelo g de Cohen, uma vez que o valor-p isolado não informa magnitude.

Métrica de risco e revisão humana. Além do acerto agregado, será contabilizado o número de trocas de rótulo (*flips*): casos individuais que mudam de resposta entre duas condições ainda que a média se mantenha. Dutta et al. (2024) medem que até 13,6% das respostas individuais trocam de rótulo entre esquemas de quantização do mesmo modelo com diferença de acurácia de até dois pontos percentuais, o que torna a média insuficiente como critério de aceitação de uma versão da base. Uma amostra das edições aceitas na base de conhecimento — até trinta por modelo, sorteadas de forma determinística e gravadas em uma planilha por modelo — será ainda avaliada segundo rubrica escrita previamente (*correta*, *parcial* ou *errada*), com os critérios fixados antes da leitura dos resultados: a rubrica é aplicada em primeira passada por um assistente de inteligência artificial, que confere cada edição contra o código da aplicação-alvo, e ratificada por um único revisor do grupo; por haver um só revisor, o resultado é reportado como proporção, sem medida de concordância.

Comparabilidade entre etapas. A comparação entre as etapas experimentais só será feita com prompts idênticos, byte a byte. Sclar et al. (2024), avaliando 320 formatos semanticamente equivalentes em cerca de 53 tarefas, medem variação de até 76 pontos de acurácia e amplitude mediana de 6,4 pontos decorrentes apenas do formato do prompt — variação que não é eliminada por exemplos no prompt, por escala do modelo nem por ajuste por instrução. Diferenças entre etapas menores que essa amplitude não seriam

atribuíveis ao desenho experimental; por isso o que não for comparável será apresentado lado a lado, e nunca como diferença testada.

4 RESULTADOS ESPERADOS

Espera-se que o Agente de QA Autônomo reduza significativamente o Tempo Médio de Reparo (MTTR) associado à manutenção de testes E2E, quando comparado à correção manual dos mesmos cenários. Projetamos que o agente identifique com precisão a causa raiz de falhas de comunicação através da depuração bidimensional. Adicionalmente, o projeto deve comprovar a viabilidade técnica da orquestração de IA em máquinas com restrições de hardware, executando o modelo integralmente em ambiente local e sem custo de infraestrutura — condição que também torna a ferramenta aplicável fora do ambiente acadêmico.

5 CRONOGRAMA DE ATIVIDADES

Etapa / Mês	Atividade Prevista
Mês 1	Revisão bibliográfica (LLMs, DOM Pruning, Contract Drift).
Mês 2	Configuração do ambiente local e da camada de inferência (Ollama com Qwen2.5-Coder quantizado), incluindo a comparação entre portes de modelo.
Mês 2 (continuação)	Comparação experimental dos modelos em três fases: prompts sem documentação (2-A), biblioteca de documentação recuperada (2-B) e biblioteca editada pelo próprio modelo em épocas atrás de validação em código (3), com análise decisória do modelo final.
Mês 3	Desenvolvimento do Módulo de Filtragem de Contexto e extração da Accessibility Tree via Playwright.
Mês 4	Codificação do fluxo de Self-Healing e lógica de interceptação do Agente.
Mês 5	Testes práticos em aplicações-alvo (injeção determinística de falhas, com Fuzzing como complemento) e coleta de métricas (MTTR e Task Success).
Mês 6	Redação, formatação ABNT e revisão dos capítulos do TCC.
Mês 7	Finalização do documento e Defesa Final perante a banca.

REFERÊNCIAS

BISWAS, S. Enhancing End-to-End Test Stability Through AI-Assisted Self-Healing: A Case Study of Playwright Healer Agent Implementation. **International Journal of Scientific Engineering and Research**, v. 14, n. 1, p. 1-12, jan. 2026. Disponível em: <https://www.researchgate.net/publication/399515915>. Acesso em: 23 set. 2026.

BOWYER, S.; AITCHISON, L.; IVANOVA, D. R. Position: Don't Use the CLT in LLM Evals With Fewer Than a Few Hundred Datapoints. In: **International Conference on Machine Learning (ICML)**, 42., 2025. Disponível em: <https://arxiv.org/abs/2503.01747>. Acesso em: 11 set. 2026.

COLLOT, S.; FRASER, C.; ZHAO, J.; SHEN, W. F.; WILLI, T.; LEONTIADIS, I. Balanced Accuracy: The Right Metric for Evaluating LLM Judges — Explained through Youden's J statistic. In: **Conference of the European Chapter of the Association for Computational Linguistics (EACL): Industry Track**, Rabat, Marrocos, 2026. p. 927-936. Disponível em: <https://aclanthology.org/2026.eacl-industry.69/>. Acesso em: 11 set. 2026.

CORRÊA, N. K.; FALK, S.; FATIMAH, S.; SEN, A.; OLIVEIRA, N. de. TeenyTinyLlama: Open-Source Tiny Language Models Trained in Brazilian Portuguese. **Machine Learning with Applications**, v. 16, art. 100558, 2024. Disponível em: <https://arxiv.org/abs/2401.16640>. Acesso em: 11 set. 2026.

CORRÊA, N. K.; SEN, A.; FATIMAH, S.; FALK, S.; LANDGRAF, L.; KASTNER, J.; FLEK, L. Tucano 2 Cool: Better Open Source LLMs for Portuguese. **arXiv preprint**, 2026. Disponível em: <https://arxiv.org/abs/2603.03543>. Acesso em: 11 set. 2026.

CUCONASU, F.; TRAPPOLINI, G.; SICILIANO, F.; FILICE, S.; CAMPAGNANO, C.; MAAREK, Y.; TONELLOTO, N.; SILVESTRI, F. The Power of Noise: Redefining Retrieval for RAG Systems. In: **International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)**, 47., Washington, DC. Nova York: ACM, 2024. p. 719-729. Disponível em: <https://arxiv.org/abs/2401.14887>. Acesso em: 11 set. 2026.

DUTTA, A.; KRISHNAN, S.; KWATRA, N.; RAMJEE, R. Accuracy is Not All You Need. In: **Advances in Neural Information Processing Systems (NeurIPS)**, 37., 2024. Disponível em: <https://arxiv.org/abs/2407.09141>. Acesso em: 11 set. 2026.

FACELI, K.; LORENA, A. C.; GAMA, J.; ALMEIDA, T. A.; CARVALHO, A. C. P. L. F. **Inteligência Artificial: uma abordagem de aprendizado de máquina**. 3. ed. Rio de Janeiro: LTC, 2025. Disponível em: <https://www.grupogen.com.br/livro-inteligencia-artificial-uma-abordagem-de-aprendizado-de-maquina-katti-faceli-ana-carolina-lorena-joao-gama-tiago-a-de-almeida-e-andre-c-p-l-f-de-carvalho-editora-ltc-9788521639206>. Acesso em: 11 set. 2026.

GARCÍA, S.; HERRERA, F. An Extension on "Statistical Comparisons of Classifiers over Multiple Data Sets" for all Pairwise Comparisons. **Journal of Machine Learning Research**, v. 9, p. 2677-2694, 2008. Disponível em: <https://www.jmlr.org/papers/volume9/garcia08a/garcia08a.pdf>. Acesso em: 11 set. 2026.

INSTATUNNEL. Automated Contract Testing: How to Detect API Drift Before It Reaches Production. **Medium**, abr. 2026. Disponível em: <https://medium.com/@instatunnel/automated-contract-testing-how-to-detect-api-drift-before-it-reaches-production-6c2a77baa2a3>. Acesso em: 29 set. 2026.

JOSEPH, R. N. Beyond LLM-Based Test Automation: A Zero-Cost Self-Healing Approach Using DOM Accessibility Tree Extraction. **Preprint**, mar. 2026. Disponível em: <https://arxiv.org/abs/2603.20358>. Acesso em: 11 set. 2026.

JÚNIOR, E.; VALEJO, A. D. B.; VALVERDE-REBAZA, J.; NEVES, V. GenIA-E2ETest: A Generative AI-Based Approach for End-to-End Test Automation. In: **Simpósio Brasileiro de Engenharia de Software (SBES)**, Recife, PE, set. 2025. Disponível em: <https://sol.sbc.org.br/index.php/sbes/article/view/37006>. Acesso em: 23 set. 2026.

KULIGOWSKI, A. Chat History and Embedding Truncation Happens Silently with No User-Visible Indication. **Issue n. 14259, repositório ollama/ollama, GitHub**, fev. 2026. Disponível em: <https://github.com/ollama/ollama/issues/14259>. Acesso em: 11 set. 2026.

LIU, N. F.; LIN, K.; HEWITT, J.; PARANJAPPE, A.; BEVILACQUA, M.; PETRONI, F.; LIANG, P. Lost in the Middle: How Language Models Use Long Contexts. **Transactions of the Association for Computational Linguistics**, v. 12, p. 157-173, 2024. Disponível em: <https://aclanthology.org/2024.tacl-1.9/>. Acesso em: 11 set. 2026.

MACIAK, T. et al. **Automated contract testing: How to detect API drift before it reaches production**. Medium, 2026.

OPITZ, J. A Closer Look at Classification Evaluation Metrics and a Critical Reflection of Common Evaluation Practice. **Transactions of the Association for Computational Linguistics**, v. 12, p. 820-836, 2024. Disponível em: <https://aclanthology.org/2024.tacl-1.46/>. Acesso em: 11 set. 2026.

SCLAR, M.; CHOI, Y.; TSVETKOV, Y.; SUHR, A. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting. In: **International Conference on Learning Representations (ICLR)**, 12., Viena, 2024. Disponível em: <https://arxiv.org/abs/2310.11324>. Acesso em: 11 set. 2026.

SHI, X.; LI, Z.; CHEN, A. R. Enhancing LLM-based Fault Localization with a Functionality-Aware Retrieval-Augmented Generation Framework. **arXiv preprint**, 2025.

SUZGUN, M.; YUKSEKGONUL, M.; BIANCHI, F.; JURAFSKY, D.; ZOU, J. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. **arXiv preprint**, 2025. Disponível em: <https://arxiv.org/abs/2504.07952>. Acesso em: 11 set. 2026.

WU, K.; WU, E.; ZOU, J. ClashEval: Quantifying the Tug-of-War Between an LLM's Internal Prior and External Evidence. In: **Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track**, 37., 2024. Disponível em: <https://arxiv.org/abs/2404.10198>. Acesso em: 11 set. 2026.

ZHANG, G.; JIANG, W.; WANG, X.; BEHR, A.; ZHAO, K.; FRIEDMAN, J.; CHU, X.; ANOUN, A. Adaptive Memory Admission Control for LLM Agents. **arXiv preprint**, 2026. Disponível em: <https://arxiv.org/abs/2603.04549>. Acesso em: 11 set. 2026.

ZHANG, J. et al. Prune4Web: DOM Tree Pruning Programming for Web Agent. In: **AAAI Conference on Artificial Intelligence**, 2026.

ZOU, W.; GENG, R.; WANG, B.; JIA, J. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In: **USENIX Security Symposium**, 34., Seattle, 2025. p. 3827-3844. Disponível em: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>. Acesso em: 11 set. 2026.

GLOSSÁRIO

Accessibility Tree (Árvore de Acessibilidade): Estrutura derivada do DOM que expõe apenas a semântica dos elementos de uma página — seu papel (*role*), nome acessível e valor —, originalmente destinada a tecnologias assistivas. Por ser consideravelmente menor que a árvore HTML completa, é utilizada neste trabalho como base para a filtragem de contexto.

Acurácia Balanceada: Média das revocações obtidas em cada classe, isto é, da proporção de acertos dentro de cada classe verdadeira, com o mesmo peso para todas elas. Diferentemente da acurácia simples, não é inflada quando o modelo passa a responder sempre o rótulo mais frequente, razão pela qual é adotada como métrica primária de comparação na Fase 3.

API (Application Programming Interface): Conjunto de definições e protocolos padronizados que permite a comunicação e a troca de dados entre diferentes sistemas de software.

Attention Dilution (Diluição de Atenção): Degradação da capacidade de raciocínio de um modelo de linguagem quando seu contexto é preenchido por grande volume de informação pouco relevante, dispersando a atenção do modelo em relação aos dados que de fato importam para a tarefa.

Contract Drift (Deriva de Contrato): Alteração silenciosa ou não documentada na estrutura dos dados retornados por uma API (como a mudança de um tipo numérico para texto), quebrando as expectativas do cliente (interface) que consome esses dados.

DOM (Document Object Model): Representação estruturada em forma de árvore do conteúdo de um documento HTML, utilizada pelos navegadores web para renderizar páginas e permitir interações via scripts.

DOM Pruning (Poda de Árvore): Técnica de redução de contexto que descarta, antes do envio ao modelo de linguagem, os nós da árvore de elementos que não são relevantes para a tarefa em análise, preservando os elementos interativos e o caminho até o elemento de interesse.

End-to-End (E2E): Abordagem de teste de software que valida o fluxo completo de uma aplicação, de ponta a ponta, simulando o comportamento de um usuário real desde a interface gráfica até a camada de banco de dados.

Época (Fase 3): Cada passagem completa do conjunto de casos pelo agente durante a Fase 3, ao final da qual as edições aprovadas pela conferência em código são consolidadas em uma nova cópia da base de conhecimento. A cópia de cada época é preservada, de modo que o desempenho possa ser lido ao longo do ciclo e não apenas no final. Não há atualização dos pesos do modelo entre épocas: o que muda é o texto da base.

FastAPI: Framework web moderno e de alto desempenho, utilizado na linguagem Python para a construção de APIs. Neste trabalho, é empregado na construção da API da aplicação-

alvo submetida aos testes.

Fertilidade (tokens por palavra): Razão entre o número de tokens produzidos pelo tokenizador e o número de palavras do texto original. Quanto maior a fertilidade, mais caro em contexto e em tempo de processamento fica o mesmo texto; para os tokenizadores de uso corrente ela é sistematicamente maior em português do que em inglês.

Fuzzing: Técnica de teste de software que envolve fornecer dados inválidos, inesperados ou aleatórios como entrada para um sistema, a fim de expor vulnerabilidades, falhas de validação ou exceções não tratadas.

LLM (Large Language Model - Modelo de Linguagem Grande): Modelo avançado de inteligência artificial treinado em vastas quantidades de texto, capaz de compreender o contexto, raciocinar e gerar linguagem natural e código de programação estruturado.

MTTR (Mean Time To Repair - Tempo Médio de Reparo): Métrica que expressa o intervalo médio entre a detecção de uma falha e a restauração verificada do funcionamento. Neste trabalho, é medido entre a detecção da falha pelo agente e a validação bem-sucedida da correção proposta.

Ollama: Runtime de código aberto para execução local de modelos de linguagem, responsável por gerenciar o download, o carregamento e a inferência dos modelos na própria máquina do usuário, sem dependência de serviços em nuvem.

Playwright / Cypress: Frameworks modernos de automação de testes para aplicações web. Ambos permitem a simulação robusta de interações de usuários e possuem capacidades nativas de interceptação de tráfego de rede no navegador.

QA (Quality Assurance - Garantia de Qualidade): Conjunto de atividades preventivas e processos de engenharia de software focados em garantir que a aplicação atenda aos requisitos especificados e aos padrões operacionais antes do lançamento em produção.

Quantização: Técnica de compressão de modelos de linguagem que reduz a precisão numérica de seus pesos, diminuindo significativamente o consumo de memória e viabilizando a inferência em hardware modesto, ao custo de uma perda controlada de precisão.

Qwen2.5-Coder: Variante de modelo de linguagem de código aberto otimizada especificamente para tarefas de programação, tradução de sintaxe, depuração e raciocínio lógico voltado para software.

RAG (Retrieval-Augmented Generation - Geração Aumentada por Recuperação): Técnica em que se recupera, de uma base de conhecimento externa, apenas o material pertinente à pergunta em questão, fornecendo-o ao modelo de linguagem junto à solicitação, em vez de depender exclusivamente do conhecimento internalizado durante o treinamento.

Self-Healing (Autocura): Capacidade intrínseca de um sistema distribuído, ou de um script de automação, de detectar ativamente falhas de execução em tempo real e aplicar

correções estruturais autonomamente para continuar operando sem interrupções.

Task Success (Sucesso da Tarefa): Métrica que expressa a proporção de cenários de falha em que o agente restaurou o fluxo de execução e o roteiro de teste concluiu sem erro, sobre o total de cenários submetidos.

Tokenização: Conversão do texto em unidades discretas — os tokens — antes de qualquer processamento pelo modelo de linguagem. É a unidade em que se contam a janela de contexto e o custo de cada inferência, e sua correspondência com caracteres ou palavras varia conforme o idioma e o tokenizador empregado.